FSCT 8561 – Security Applications Midterm

Operation Sentinel: Secure Server Assessment & Client Security Application

Billy Zandan A01371854

GitHub Repo link: https://github.com/wotprizzleR6/FSCT8561-Midterm-Billy-Zandan.git

Day 1 — Architecture and Threat Modeling (25%)

Part A — Architecture Design (10%)

1. System Architecture Diagram

The diagram below shows each Python file's role, and the flow of data passed between files.

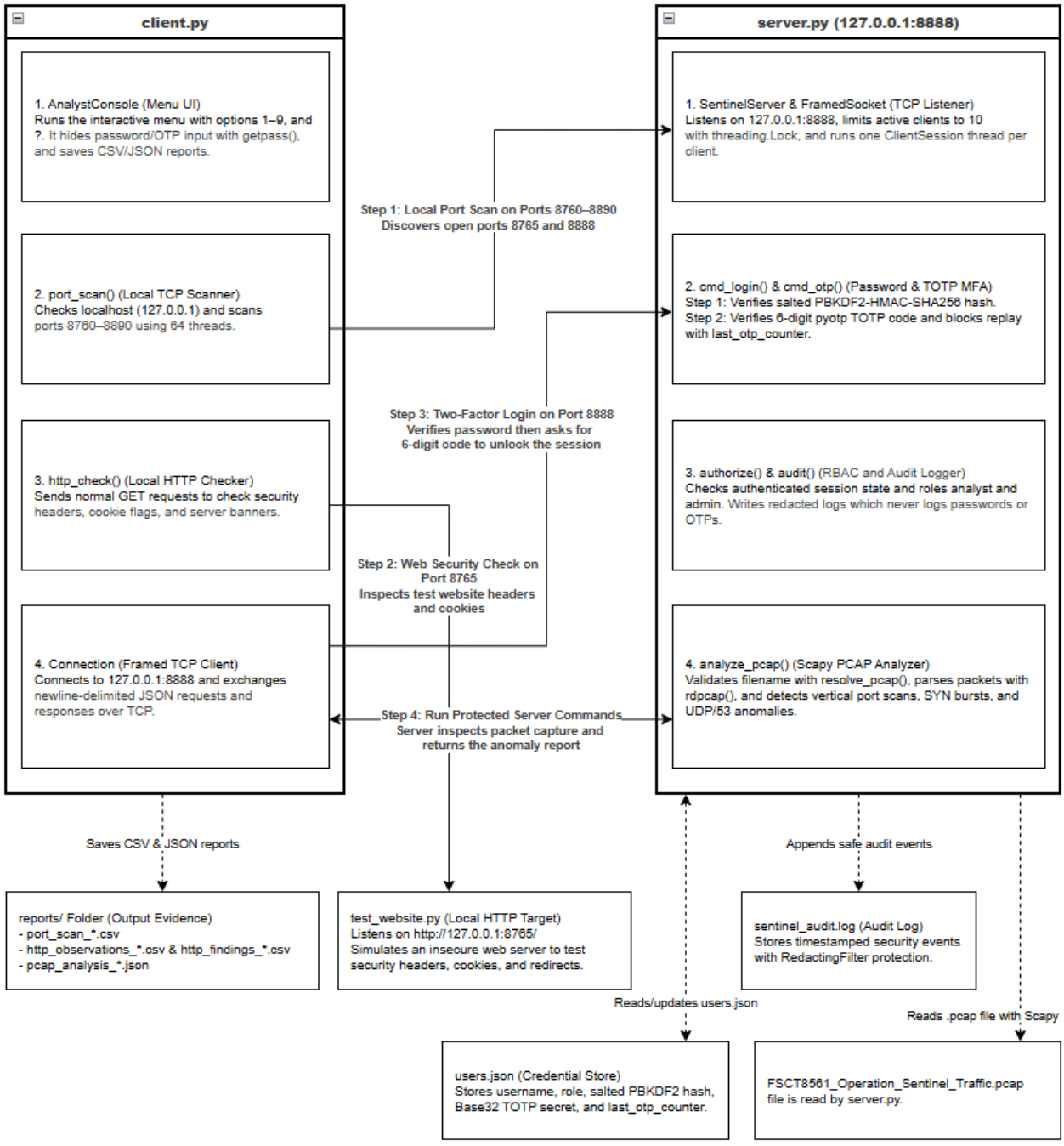


Figure 1: System Architecture Diagram

Rationale for a Stateful Interaction Model

This system uses a stateful TCP Model for each connection thread with the ClientSession. It requires a stateful design because the authentication is performed in two stages (CONNECTED to OTP_PENDING to AUTHENTICATED). To accept a 6 digit TOTP token the server has to know that the client has already passed the password verification. Also, maintaining authenticated state on persistent TCP sockets avoids sending sensitive credentials with every successive operation, and imposes a 30 minute session timeout. Stateless paradigm means re-transmitting passwords in every request and exposing credentials .

Part B — Adversarial Threat Model (15%)

The table below defines five threats across sockets, port scanning, authentication/MFA, Scapy PCAP analysis, and HTTP checking.

Asset	Attack Vector	Impact	Mitigation Implemented	Residual Risk
T-01: TCP Sockets (127.0.0.1:8888)	Local packet sniffing or session injection.	Credential theft and command session hijacking.	Bound strictly to 127.0.0.1.	Plain TCP lacks encryption. Full protection requires TLS 1.3.
T-02: Port Scanner (client.py)	Scanning external IPs or spawning thousands of threads can exhaust the sockets.	Unauthorized network probing and exhaustion of host resources.	Enforces localhost scope, caps range and limits the pool to 64 threads.	Full TCP handshakes still log connection events on target services.
T-03: Auth & MFA (server.py & users.json)	Password brute-forcing, timing attacks, or TOTP replay.	Unauthorized access to PCAP_ANALYZE or DIAG.	Salted PBKDF2 with constant-time comparisons, lockout after five attempts, and a replay counter.	Intercepted OTP could be replayed within the valid 30 second window.
T-04: Scapy PCAP Analyzer (server.py)	Path traversal (../../) or memory exhaustion with huge files.	Unauthorized file access and/or server memory crash (DoS).	Strict filename regex, path containment check, and a 50 MB file cap.	Scapy rdpcap() still loads the full capture into memory.
T-05: HTTP Checker (client.py)	Hanging connections, redirect loops, or SSRF scope drift.	Client lockup or misleading security findings.	Restricts host to localhost, disables redirect following, and enforces a 5 second timeout.	Passive checks identify header gaps, not active exploits like SQL injection.

Day 2 — Protocol and Component Specification (25%)

Part C — Protocol Design (15%)

1. Message Structure

- Framing: Newline-delimited UTF-8 JSON (\n) over standard TCP stream sockets.
- Stream Buffering: Client and server buffers incoming TCP chunks until \n to prevent packet fragmentation errors.
- Payload Limits: Requests are capped at 16 KB and responses are capped at 4 MB to prevent denial-of-service.
- Wire Format:
    - Request in client.py: frame = {"v": 1, "cmd": cmd, "args": args or {}, "req_id": self._req_id}
    - Response in server.py: return {"v": PROTOCOL_VERSION, "status": status, "code": code, "message": message, "data": data or {}}

2. Authentication and MFA Sequence

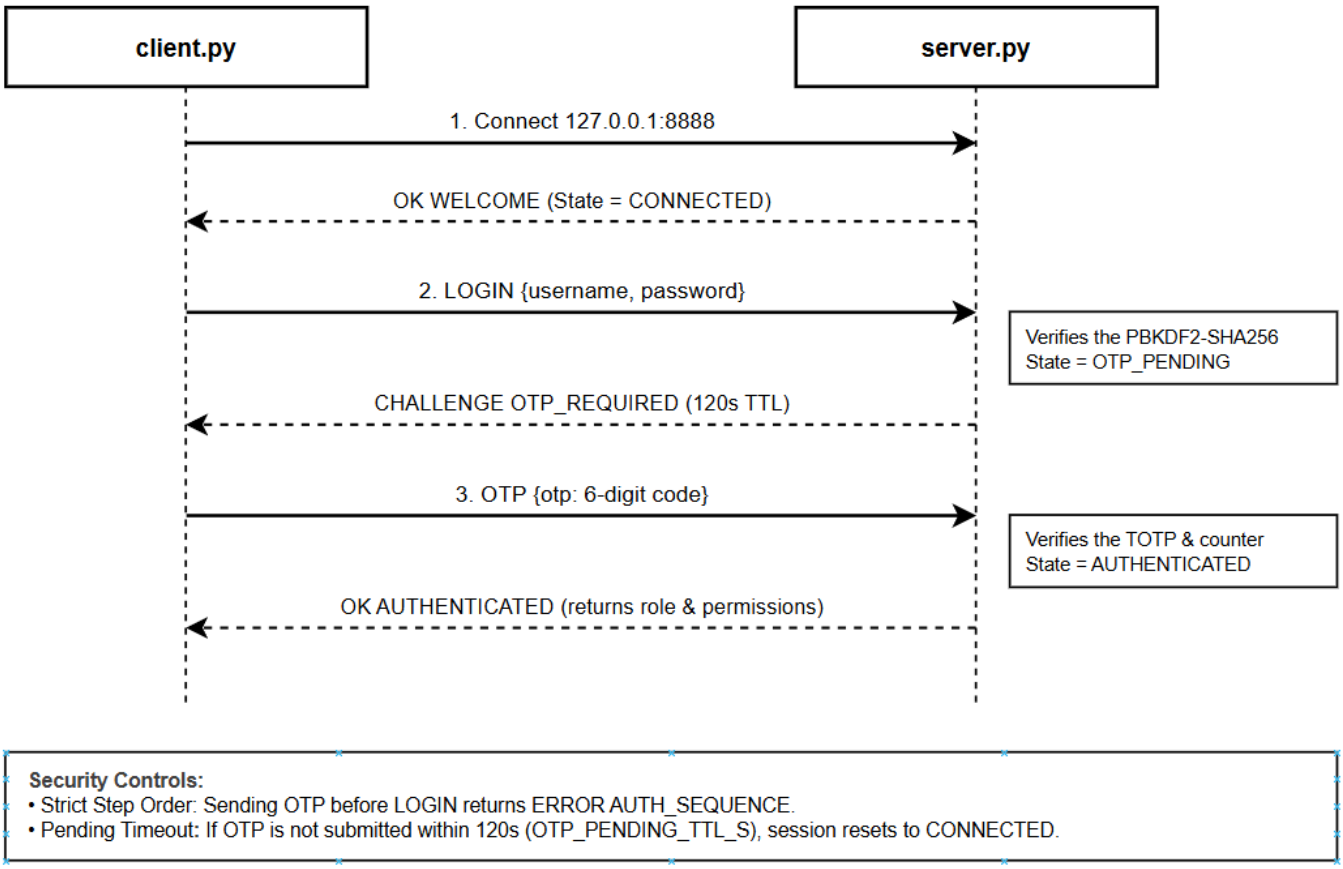


Figure 2: Two-Factor Authentication Protocol Sequence

3. Authorization Rules for Protected Commands

Command	Required State	Allowed Roles	Behavior & Security Rule
PING, HELP, EXIT	Any State	Public	Checks server liveness, lists commands, or closes the connection.
LOGIN	CONNECTED	Public	Verifies username and password, transitions state to OTP_PENDING.
OTP	OTP_PENDING	Public	Verifies 6-digit TOTP code, transitions state to AUTHENTICATED.
WHOAMI, LOGOUT	AUTHENTICATED	analyst and admin	Returns session metadata or resets authentication to CONNECTED.
PCAP_ANALYZE	AUTHENTICATED	analyst and admin	Runs offline Scapy analysis on the specified PCAP file.
DIAG	AUTHENTICATED	restricted to admin only	Returns server diagnostics and rejects analyst role with ERROR FORBIDDEN.

4. Malformed Request Handling and Error Responses

- Frame and Schema Errors: Invalid UTF-8 returns BAD_FRAME, invalid JSON returns BAD_JSON, and any missing/extra keys return BAD_REQUEST.
- Malformed Abuse Cutoff: Five malformed requests (MAX_MALFORMED_PER_SESSION = 5) trigger an automatic socket disconnect.
- Uniform Auth Errors: Unknown users, wrong passwords, and locked accounts all return error AUTH_FAILED to prevent user enumeration.

5. Session Termination

- Graceful Exit/Logout: EXIT sends BYE GOODBYE and closes the socket shutdown(socket.SHUT_RDWR) and LOGOUT drops auth back to CONNECTED.
- Timeouts and Lockouts: Sockets close after 300s idle (IDLE_TIMEOUT_S) or 3 failed logins and auth expires after 1800 seconds (SESSION_MAX_AGE_S).

6. Replay and Security Limitations: Implemented vs. Future Deployment

Security Control	Implemented Now	Proposed for Future Production
Credential Hashing	Salted PBKDF2-HMAC-SHA256 with hmac.compare_digest().	Store hashes and TOTP keys in an HSM-backed database vault.
OTP Replay Guard	Stores last_otp_counter in users.json so that used TOTP codes are rejected.	Bind MFA challenges to TLS session keys or FIDO2 tokens.
Transport Security	Unencrypted plain TCP on 127.0.0.1 with explicit startup warning.	Wrap sockets in TLS 1.3 (ssl.SSLContext) with mutual TLS (mTLS).

Part D — Component Design (10%)

1. Port Scanner Component (client.py)

- Inputs/Outputs: Takes localhost (127.0.0.1), port range (8760-8890), and engine (socket/nmap). Then outputs console table and port_scan_*.csv.
- Expected Behavior: Uses ThreadPoolExecutor(max_workers=64) with socket.connect_ex() and resolves services with getservbyport().
- Failure Conditions: Non-loopback hosts raise ScopeError, port ranges exceeding 1024 raise ValueError, and unresponsive ports time out after 2.5 seconds on Windows.
- Security Controls: Enforces 127.0.0.1 localhost, caps worker threads at 64, and notes that open ports show reachability, not a vulnerability.

2. Multi-Factor Authentication Module (server.py)

- Inputs/Outputs: Takes username, password, and 6-digit otp, then outputs session state transition (AUTHENTICATED), user role, and audit logs.
- Expected Behavior: Verifies salted PBKDF2 hash in cmd_login(), then checks pyotp.TOTP (+/- 1 step) and consume_otp_counter() in cmd_otp().
- Failure Conditions: Bad credentials or replayed OTPs return AUTH_FAILED; three session failures close the socket and five failures lock the account for 120 seconds.
- Security Controls: Constant-time hmac.compare_digest(), dummy PBKDF2 hash for unknown users, last_otp_counter replay block, and del password cleanup.

3. HTTP Security Configuration Checker (client.py)

- Inputs/Outputs: Takes base URL (http://127.0.0.1:8765/) and paths (/, /health, /redirect, /missing), then outputs http_observations_*.csv and http_findings_*.csv under the reports folder.
- Expected Behavior: Sends GET requests via requests.Session() with allow_redirects=False and checks CSP, nosniff, X-Frame-Options, Referrer-Policy, cookies, and banners.
- Failure Conditions: Catches ConnectionError and Timeout cleanly, recording CONNECTION_FAILED and marking the check as Inconclusive.
- Security Controls: Validates loopback URL scope, disables redirects, de-duplicates findings (FindingSet), and labels findings as configuration review only.

4. Offline PCAP Analyzer (server.py)

- Inputs/Outputs: Takes the provided the .pcap file, then outputs JSON summary (hashes, protocols, handshakes) and a findings list.

- Expected Behavior: Parses packets with Scapy rdpcap() and runs sliding-window detectors (_max_in_window) for scans, SYN bursts, UDP/53 mismatches, and rate spikes.
- Failure Conditions: Path traversal or bad names return BAD_REQUEST, and missing files return NOT_FOUND, and files over 50 MB return TOO_LARGE.
- Security Controls: Regex filename allow-list (resolve_pcap), a 50 MB cap, (path, mtime, size) result cache, and ASCII-sanitized payload previews.

5. Sensitive Information Intentionally Avoided in Logs

- Excluded Category: Plaintext passwords, Base32 TOTP secrets, 6-digit OTP codes, and unknown/unenrolled usernames are never written to sentinel_audit.log.
- Justification 1: Logging passwords or TOTP secrets would let anyone reading log files or backups hijack accounts.
- Justification 2: Users often type passwords into the username prompt, so failed unknown usernames are logged as <unenrolled>.
- Justification 3: RedactingFilter sanitizes any password=, otp=, or secret= pattern to [REDACTED] before writing logs.

Day 3 — Implementation, Testing, and Analysis (50%)

Part E — Full Integrated Implementation (30%)

1. Framed TCP Client and Server (client.py, server.py): Uses Connection and FramedSocket over 127.0.0.1:8888 with \n delimited JSON framing taken from labs zero and one.
2. Password Hashing and TOTP MFA (server.py): Uses salted PBKDF2-HMAC-SHA256 and pyotp 6-digit TOTP codes taken from lab 3 and class textbook Ortega Ch. 14.
3. Authorization and Role Enforcement (server.py): authorize() blocks unauthenticated commands (AUTH_REQUIRED) and restricts DIAG to admin (FORBIDDEN).
4. Local TCP Port Scanner (client.py): port_scan() scans 127.0.0.1:8760-8890 using 64 threads (connect_ex or nmap -sT -Pn), finding 8765 and 8888 open taken from lab two.
5. Local HTTP Security Checker (client.py): http_check() inspects http://127.0.0.1:8765/ for missing security headers, insecure cookies, and server banners taken from lab five.
6. Offline Scapy PCAP Analysis (`server.py`): analyze_pcap() parses FSCT8561_Operation_Sentinel_Traffic.pcap and flags six traffic anomalies taken from lab four.
7. Server-to-Client PCAP Transmission (client.py, server.py): PCAP_ANALYZE sends the full JSON report to client.py, which prints tables and saves a pcap_analysis_*.json.
8. Validation, Error Handling, and Safe Logging: Validates all inputs, handles network failures cleanly, and logs events to sentinel_audit.log without secrets.



Figure 3: Enroll Analyst, server listening on 127.0.0.1:8888, execute client.py for menu

Part F — Comprehensive Testing (15%)

All 14 automated tests in run_tests.py ran against live test_website.py (:8765) and server.py (:8888) instances.

Test ID	Test Case	Input/Action	Expected Result	Actual Result	Status	Captured Output
TEST 1	2FA Login Success	LOGIN (analyst + pass) then valid 6-digit OTP	OTP_REQUIRED then AUTHENTICATED	OTP_REQUIRED -> AUTHENTICATED (analyst)	Pass	event=AUTH_SUCCESS user="analyst" role="analyst" mfa="totp"
TEST 2	Failed Password	LOGIN with analyst and WrongPassword999!	ERROR AUTH_FAILED (attempts_remaining: 2)	ERROR AUTH_FAILED (attempts_remaining: 2)	Pass	event=AUTH_FAIL stage="password" reason="bad_password"
TEST 3	Failed TOTP Code	Valid password then invalid OTP 000000	ERROR AUTH_FAILED; state reset	ERROR AUTH_FAILED; state=CONNECTED	Pass	event=AUTH_FAIL stage="otp" reason="bad_otp"
TEST 4	Unauthenticated Reject	Send PCAP_ANALYZE before LOGIN	ERROR AUTH_REQUIRED	ERROR AUTH_REQUIRED	Pass	event=ACCESS_DENIED cmd="PCAP_ANALYZE" reason="unauthenticated"
TEST 5	Role Check (RBAC)	Analyst role sends admin command DIAG	ERROR FORBIDDEN	ERROR FORBIDDEN	Pass	event=ACCESS_DENIED role="analyst" cmd="DIAG" reason="insufficient_role"
TEST 6	Open Port Scan	Scan 127.0.0.1 ports 8760-8890	Ports 8765 and 8888 reported open	open_ports_found: 8765, 8888	Pass	8765 open test website; 8888 open Sentinel server
TEST 7	Closed Port Scan	Scan inactive port 127.0.0.1:8766	Port 8766 reported closed	port: 8766, state: closed	Pass	connect_ex() returned WSAECONNREFUSED (10061)
TEST 8	HTTP Security Check	Check http://127.0.0.1:8765/ (4 paths)	Flags missing headers, cookie flags, banner	reachable: true, findings: 10	Pass	[HTTP-01] Missing CSP; [HTTP-02] Cookie lacks HttpOnly
TEST 9	HTTP Failure Handling	Check stopped port http://127.0.0.1:8999/	reachable: False, CONNECTION_FAILED	reachable: false, CONNECTION_FAILED	Pass	Caught ConnectionError cleanly; status=Inconclusive
TEST 10	PCAP Threat Detection	Send PCAP_ANALYZE on default .pcap	123 packets; flags scan, burst, UDP/53	packet_count: 123, findings: 6	Pass	[PCAP-01] Port scan; [PCAP-02] UDP/53; [PCAP-03] SYN burst
TEST 11	Malformed Input	Send 'NOT_JSON' and {'injected': 1}	Returns BAD_JSON and BAD_REQUEST	BAD_JSON and BAD_REQUEST	Pass	event=MALFORMED_REQUEST count="1" & count="2"
TEST 12	OTP Replay Blocked	Reuse same 6-digit OTP in second session	Session 1: OK; Session 2: AUTH_FAILED	Session 1: OK; Session 2: AUTH_FAILED	Pass	event=OTP_REPLAY_BLOCKED reason="otp_replay"
TEST 13	Clean Disconnection	Send EXIT command to server	Server returns BYE GOODBYE & closes	status: BYE, code: GOODBYE	Pass	event=DISCONNECT reason="client_exit"
TEST 14	Audit Log Redaction	Scan sentinel_audit.log for secrets	Zero passwords or OTPs in log file	leaks_detected: [] across all entries	Pass	0 occurrences of Cyber123! or TOTP secrets

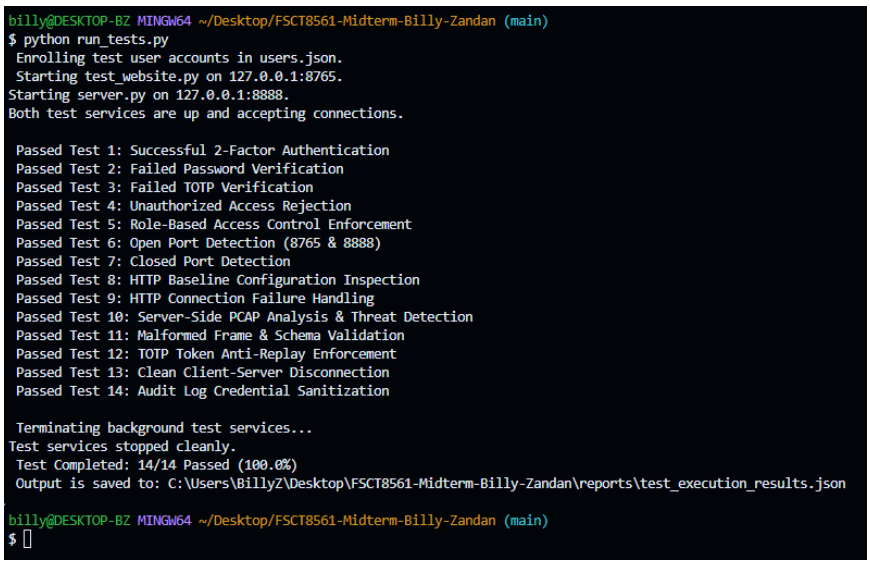


Figure 4: Execute all 14 tests with run_tests.py

```
billy@DESKTOP-BZ MINGW64 ~/Desktop/FSCT8561-Midterm-Billy-Zandan (main)
$ python client.py

Console

This is localhost only and plain TCP transport is not encrypted.
OK WELCOME: Operation Sentinel server. Send HELP for commands.
    Connected to 127.0.0.1:8888 (framing: newline-delimited JSON, encrypted: False)
Server: 127.0.0.1:8888 CONNECTED | Auth: not authenticated
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select> 2
Username: analyst
Password:
ERROR AUTH_FAILED: Authentication failed.
    Attempts remaining on this connection: 2
Server: 127.0.0.1:8888 CONNECTED | Auth: not authenticated
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select> 2
Username: analyst
Password:
CHALLENGE OTP_REQUIRED: Password accepted. Submit your 6-digit TOTP code.
6-digit TOTP code:
OK AUTHENTICATED: Welcome analyst. MFA complete.
    Role: analyst   Permissions: LOGOUT, PCAP_ANALYZE, WHOAMI
Server: 127.0.0.1:8888 CONNECTED | Auth: analyst (analyst)
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select>
```

Figure 5: Demonstrate failed authentication and successful authentication

```
billy@DESKTOP-BZ MINGW64 ~/Desktop/FSCT8561-Midterm-Billy-Zandan (main)
$ python server.py
2026-10-23 18:16:03,658 | INFO    | MainThread | event=SERVER_START host="127.0.0.1" port="8888" users="5" pcap_dir="FSCT8561-Midterm-Billy-Zandan" transport="plain_tcp"
Sentinel/1.0 listening on 127.0.0.1:8888  (Ctrl+C to stop)
Transport is plain TCP (not encrypted) - localhost testing only.
2026-10-23 18:16:06,268 | INFO    | client-1 | event=CONNECT session="1" peer="127.0.0.1:28261"
2026-10-23 18:16:32,555 | WARNING | client-1 | event=AUTH_FAIL session="1" peer="127.0.0.1:28261" stage="password" user="analyst" reason="bad_password" attempt="1/3"
2026-10-23 18:16:47,172 | INFO    | client-1 | event=AUTH_PASSWORD_OK session="1" peer="127.0.0.1:28261" user="analyst"
2026-10-23 18:16:55,054 | INFO    | client-1 | event=AUTH_SUCCESS session="1" peer="127.0.0.1:28261" user="analyst" role="analyst" mfa="totp"
```

Figure 5-1: Server-side

```
billy@DESKTOP-BZ MINGW64 ~/Desktop/FSCT8561-Midterm-Billy-Zandan (main)
$ python client.py

Console

This is localhost only and plain TCP transport is not encrypted.
OK WELCOME: Operation Sentinel server. Send HELP for commands.
    Connected to 127.0.0.1:8888 (framing: newline-delimited JSON, encrypted: False)
Server: 127.0.0.1:8888 CONNECTED | Auth: not authenticated
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select> 5
PCAP file on server [FSCT8561_Operation_Sentinel_Traffic.pcap]:
Requesting server-side analysis ...
ERROR AUTH_REQUIRED: Authentication (password + OTP) required.
    (request rejected by the server's authorization check)
Server: 127.0.0.1:8888 CONNECTED | Auth: not authenticated
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select> 2
Username: analyst
Password:
CHALLENGE OTP_REQUIRED: Password accepted. Submit your 6-digit TOTP code.
6-digit TOTP code:
OK AUTHENTICATED: Welcome analyst. MFA complete.
    Role: analyst   Permissions: LOGOUT, PCAP_ANALYZE, WHOAMI
Server: 127.0.0.1:8888 CONNECTED | Auth: analyst (analyst)
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select> 6
ERROR FORBIDDEN: Role 'analyst' is not permitted to run DIAG.
    (request rejected by the server's authorization check)
Server: 127.0.0.1:8888 CONNECTED | Auth: analyst (analyst)
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select>
```

```
billy@DESKTOP-BZ MINGW64 ~/Desktop/FSCT8561-Midterm-Billy-Zandan (main)
$ python server.py
2026-10-23 19:31:21,985 | INFO    | MainThread | event=SERVER_START host="127.0.0.1" port="8888" users="5" pcap_dir="FSCT8
561-Midterm-Billy-Zandan" transport="plain_tcp"
Sentinel/1.0 listening on 127.0.0.1:8888  (Ctrl+C to stop)
Transport is plain TCP (not encrypted) - localhost testing only.
2026-10-23 19:31:35,986 | INFO    | client-1 | event=CONNECT session="1" peer="127.0.0.1:14748"
2026-10-23 19:32:00,715 | WARNING | client-1 | event=ACCESS_DENIED session="1" peer="127.0.0.1:14748" cmd="PCAP_ANALYZE" r
eason="unauthenticated" state="CONNECTED"
2026-10-23 19:33:27,059 | INFO    | client-1 | event=AUTH_PASSWORD_OK session="1" peer="127.0.0.1:14748" user="analyst"
2026-10-23 19:33:41,922 | INFO    | client-1 | event=AUTH_SUCCESS session="1" peer="127.0.0.1:14748" user="analyst" role="
analyst" mfa="totp"
2026-10-23 19:34:13,671 | WARNING | client-1 | event=ACCESS_DENIED session="1" user="analyst" role="analyst" cmd="DIAG" re
ason="insufficient_role"
```

Figure 5-2: Unauthorized and RBAC Rejection

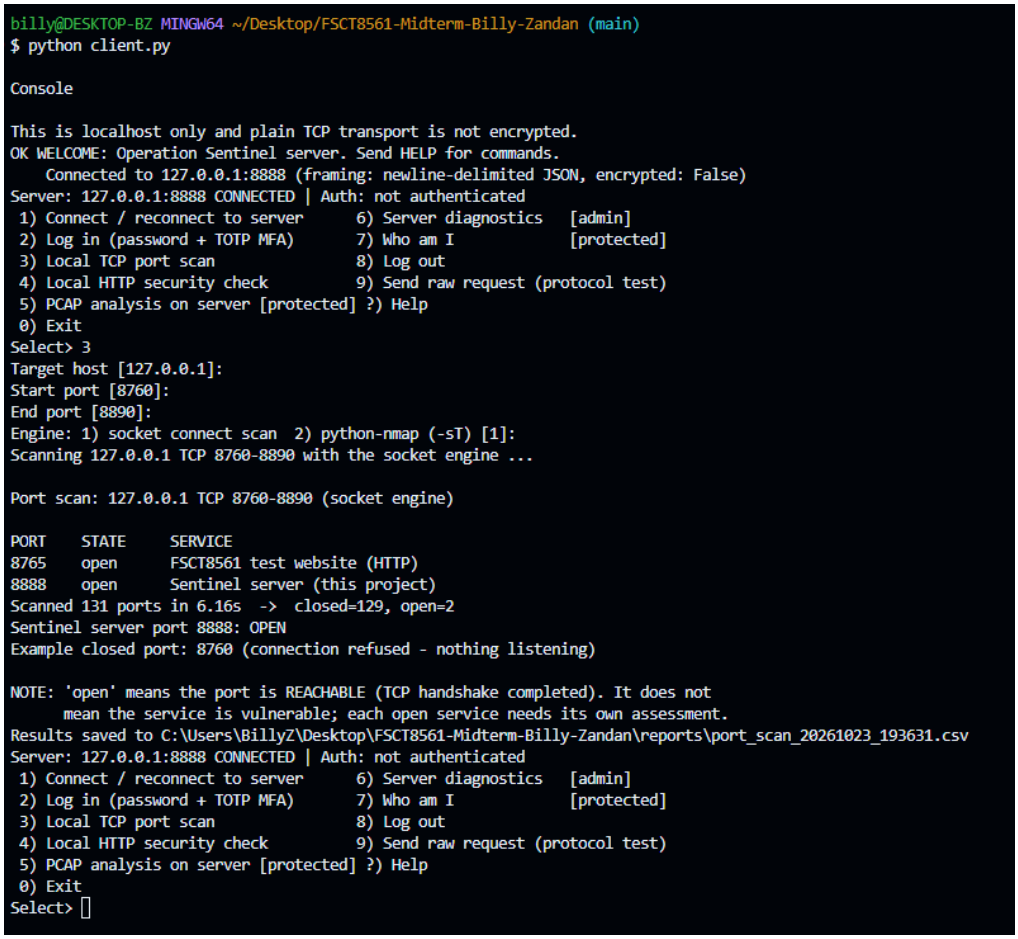


Figure 5-3: Local TCP Port Scan Open & Closed



Figure 5-4: HTTP Check on http://127.0.0.1:8765/

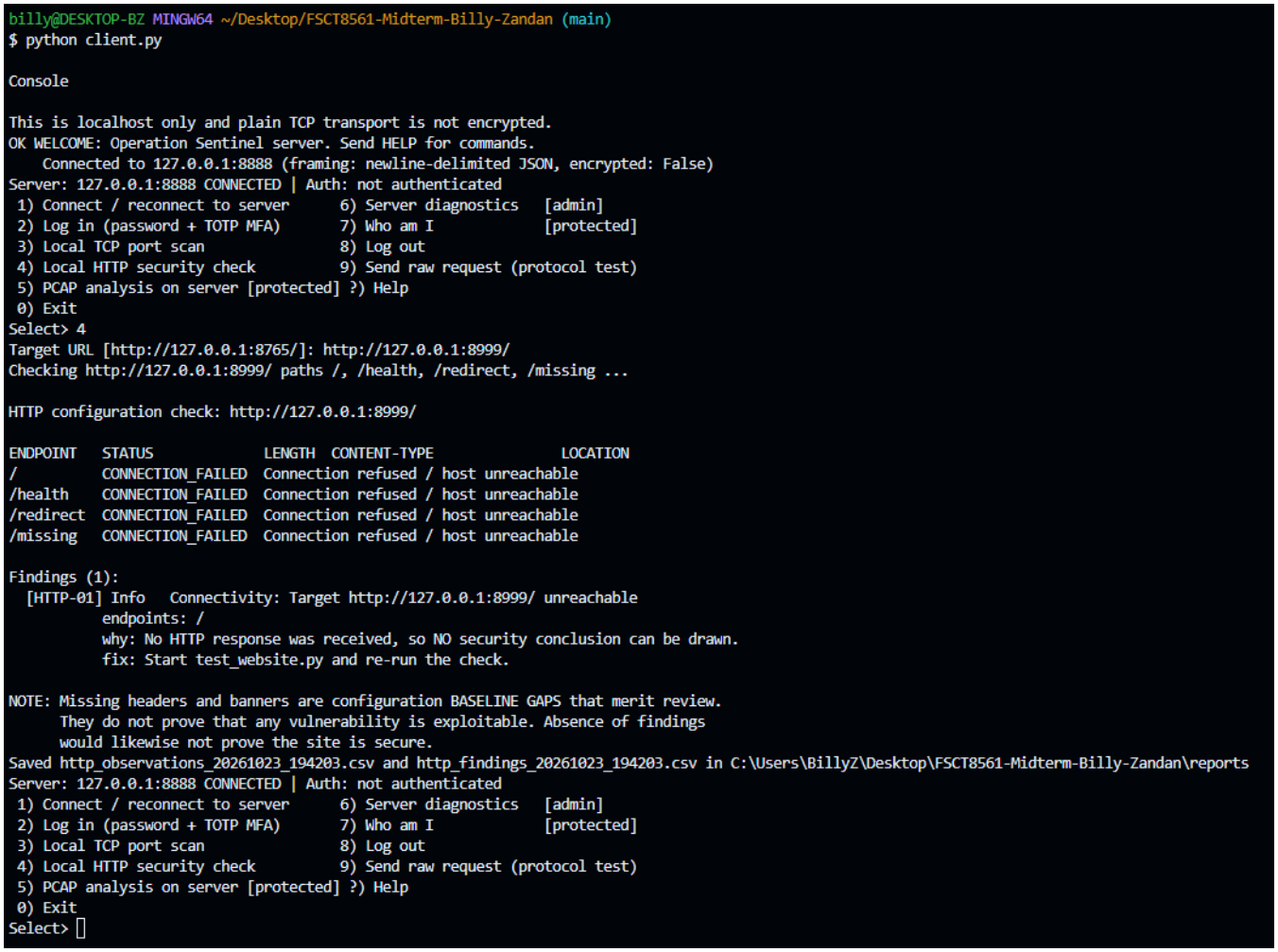


Figure 5-5: HTTP Connection Failure on http://127.0.0.1:8999/



Figure 5-6: Server-Side PCAP Analysis

```
billy@DESKTOP-BZ MINGW64 ~/Desktop/FSCT8561-Midterm-Billy-Zandan (main)
$ python client.py

Console

This is localhost only and plain TCP transport is not encrypted.
OK WELCOME: Operation Sentinel server. Send HELP for commands.
    Connected to 127.0.0.1:8888 (framing: newline-delimited JSON, encrypted: False)
Server: 127.0.0.1:8888 CONNECTED | Auth: not authenticated
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select> 9
Protocol test mode: the line is sent EXACTLY as typed (no JSON wrapping).
Example: {"v":1,"cmd":"PCAP_ANALYZE","args":{}}   or   not-json
raw> not-json
{
  "v": 1,
  "status": "ERROR",
  "code": "BAD_JSON",
  "message": "Frame is not valid JSON.",
  "data": {}
}
Server: 127.0.0.1:8888 CONNECTED | Auth: not authenticated
 1) Connect / reconnect to server      6) Server diagnostics   [admin]
 2) Log in (password + TOTP MFA)       7) Who am I             [protected]
 3) Local TCP port scan                8) Log out
 4) Local HTTP security check          9) Send raw request (protocol test)
 5) PCAP analysis on server [protected] ?) Help
 0) Exit
Select> 0
BYE GOODBYE: Goodbye.
Disconnected.

billy@DESKTOP-BZ MINGW64 ~/Desktop/FSCT8561-Midterm-Billy-Zandan (main)
$
```

```
PS C:\Users\BillyZ\Desktop\FSCT8561-Midterm-Billy-Zandan> Get-Content -Tail 15 sentinel_audit.log
2026-10-23 19:41:32,142 | INFO    | client-3 | event=DISCONNECT session="3" peer="127.0.0.1:36060" user="-" reason="client_exit" duration_s="97.6"
2026-10-23 19:41:36,646 | INFO    | client-4 | event=CONNECT session="4" peer="127.0.0.1:39803"
2026-10-23 19:45:26,147 | INFO    | client-4 | event=EXIT session="4" user="-"
2026-10-23 19:45:26,147 | INFO    | client-4 | event=DISCONNECT session="4" peer="127.0.0.1:39803" user="-" reason="client_exit" duration_s="229.5"
2026-10-23 19:45:28,545 | INFO    | client-5 | event=CONNECT session="5" peer="127.0.0.1:57137"
2026-10-23 19:45:40,160 | INFO    | client-5 | event=AUTH_PASSWORD_OK session="5" peer="127.0.0.1:57137" user="analyst"
2026-10-23 19:45:50,499 | INFO    | client-5 | event=AUTH_SUCCESS session="5" peer="127.0.0.1:57137" user="analyst" role="analyst" mfa="totp"
2026-10-23 19:46:04,160 | INFO    | client-5 | event=PCAP_ANALYZE_START session="5" user="analyst" file="FSCT8561_Operation_Sentinel_Traffic.pcap"
2026-10-23 19:46:04,176 | INFO    | client-5 | event=PCAP_ANALYZE_DONE session="5" user="analyst" file="FSCT8561_Operation_Sentinel_Traffic.pcap" packets="123" findings="6" cached="False"
2026-10-23 19:48:40,689 | INFO    | client-5 | event=EXIT session="5" user="analyst"
2026-10-23 19:48:40,689 | INFO    | client-5 | event=DISCONNECT session="5" peer="127.0.0.1:57137" user="analyst" reason="client_exit" duration_s="192.1"
2026-10-23 19:48:43,294 | INFO    | client-6 | event=CONNECT session="6" peer="127.0.0.1:29020"
2026-10-23 19:48:55,548 | WARNING | client-6 | event=MALFORMED_REQUEST session="6" peer="127.0.0.1:29020" code="[REDACTED]" count="1"
2026-10-23 19:49:11,395 | INFO    | client-6 | event=EXIT session="6" user="-"
2026-10-23 19:49:11,396 | INFO    | client-6 | event=DISCONNECT session="6" peer="127.0.0.1:29020" user="-" reason="client_exit" duration_s="28.1"
PS C:\Users\BillyZ\Desktop\FSCT8561-Midterm-Billy-Zandan>
```

Figure 5-7: Malformed Input, Disconnect & Audit Log

Part G — Security Analysis and Reflection (5%)

Question 9: Which threats from your Day 1 model did your implementation mitigate, and which remain?

- Mitigated (T-02 to T-05): Loopback checks, 64-thread scan caps, salted PBKDF2 (600,000 iterations), last_otp_counter anti-replay, resolve_pcap(), and HTTP timeouts mitigated T-02 to T-05.
- Other Risks (T-01, T-02) T-01: Plain TCP permits local sniffing T-02: connect_ex() searches log connections on target services.

Question 10: What suspicious traffic did your PCAP analyzer identify? Cite packet-level or summary evidence.

- The FSCT8561_Operation_Sentinel_Traffic.pcap analysis includes 123 packets, with 113 TCP, and 10 UDP packets coming from 4 source IPs.

1. Vertical Port Scan (PCAP-01, High): 192.0.2.50 delivered 20 unanswered TCP SYNs to 192.0.2.10 ports 20-39 (packets #82-101).

2. SSH SYN Burst (PCAP-03, Medium): 192.0.2.60 transmitted 16 unanswered TCP SYNs (packets #102-117) to 192.0.2.10:22.
3. UDP/53 Protocol Mismatch (PCAP-02, Medium): 10 non-DNS UDP packets (#72-81) with raw ASCII traffic were transmitted to 192.0.2.10:53 from 192.0.2.20.
4. Half-Open Probes (PCAP-06, Low): 192.0.2.20 sent 5 unanswered TCP SYNs (packets #118-122) to 192.0.2.10:80.
5. Packet Rate Spikes (PCAP-04, PCAP-05, Low): 20 and 16 packets arrived in single 1.0 second windows, corresponding to the scan and the SYN burst.

Question 11: What did your HTTP checker find, and why are its findings not proof of exploitability?

- Findings on http://127.0.0.1:8765/: Missing Content-Security-Policy, X-Frame-Options, Referrer-Policy, and X-Content-Type-Options: nosniff, cookie training_session did not have HttpOnly, Secure, and SameSite. Server: FSCT8561LabHTTP/1.0 revealed.
- Why It's Not Proof of Exploitability: Security headers and cookie flags are browser defense-in-depth safeguards, not active bugs in themselves.
- Required Root Flaw to Exploit: Missing CSP or HttpOnly cannot be used for XSS unless the web app has unescaped script injection bug as well.

Question 12: What are the confidentiality and replay limitations of a plain TCP connection, even with TOTP MFA?

- No Confidentiality: Plain TCP transmits usernames, passwords, 6-digit OTPs, and PCAP reports in cleartext and local network sniffers can read them.
- Post-Authentication Session Hijacking: TOTP merely checks the login, therefore an active attacker can inject faked TCP packets onto the open socket once authentication is done.
- Pre-Delivery OTP Interception: If an attacker intercepts an OTP packet sent by a client before it reaches the server, they can easily re-use the code within 30 seconds.

Question 13: What would you improve before deploying this application in production?

1. TLS 1.3 and mTLS: Wrap the sockets using ssl.SSLContext(ssl.PROTOCOL_TLSv1_3) with client certificates to prevent sniffing and hijacking.
2. Streaming PCAP Parser: Switch from rdpcap() to Scapy PcapReader to stream multi-gigabyte captures in memory.
3. Database & Secret Vault: Move credentials out of users.json into PostgreSQL, with TOTP secrets protected by a hardware key vault.
4. Asyncio Server and Web Hardening: Use asyncio for scalable sockets, add HTTPS/security headers on the web app and ship logs to a remote SIEM.